# Concept Questions - Answers

## Week 8 Practice - Concept Questions 1-5

1. Encapsulation and email validation
The User object should keep email private and own its validation. A changeEmail(newEmail) method checks format and uniqueness before updating the field. Direct unrestricted modification could introduce malformed or duplicate email values. If validation fails, the old valid email remains unchanged.
2. Inheritance versus composition for report features
Inheritance is useful for a real is-a specialization. DataExport and DataVisualization are optional capabilities that vary independently, so composition is more flexible: a report can combine either, both, or neither. A single inheritance hierarchy for every combination causes rigid, combinatorial subclasses and makes features harder to extend.
3. Abstraction, overriding, and runtime polymorphism
Define a Notification interface with send(). EmailNotification, SMSNotification, PushNotification, and InAppNotification override that method. NotificationService depends on the interface and calls send() without checking concrete types. At runtime Java invokes the actual object’s implementation, so adding InAppNotification does not require rewriting the service.
4. Composition and aggregation
Composition is strong ownership and usually a shared lifecycle; aggregation is weaker ownership where a part may survive independently. Organization-Department is composition here because the organization’s end dissolves departments. Department-Employee is aggregation because employees survive a department’s dissolution and can move or be reassigned.
5. Student-Course multiplicity and unique enrollment
The relationship is many-to-many: one Student may take many Courses and one Course may have many Students. Multiplicity describes how many instances participate; it does not forbid duplicate enrollment records for one pair. Enforce the rule with a unique constraint on (studentId, courseId) in an Enrollment table/class or validation in the enrollment service.